

Chapter - 1

Introduction To Computer, Program, and Java

The term programming means to create (or develop) software, which is also called a program. In basic terms, software contains the instructions that tell a computer—or a computerized device—what to do.

What is Computer?

A computer is an electronic device that stores and processes data.

A computer consists of the following major hardware components (as shown in Figure 1.1)

- A central processing unit (CPU)
- Memory (main memory)
- Storage devices (such as disks and CDs)
- Input devices (such as the mouse and keyboard)
- Output devices (such as monitors and printers)
- Communication devices (such as modems and network interface cards)

A computer's components are interconnected by a subsystem called a bus. Which is built in computer's motherboard.

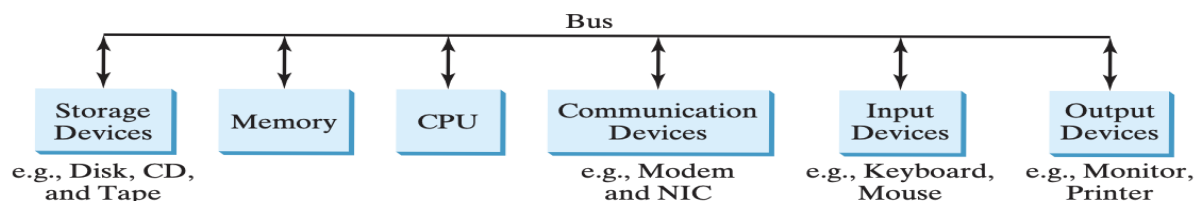


FIGURE 1.1 A computer consists of a CPU, memory, storage devices, input devices, output devices, and communication devices.

CPU: The central processing unit (CPU) is the computer's brain. It retrieves instructions from memory and executes them. The CPU usually has two components: a *control unit* and an *arithmetic/logic unit*. The control unit *controls and coordinates the actions* of the other components. The arithmetic/logic unit *performs numeric operations* (addition, subtraction, multiplication, division) and logical operations (comparisons).

Core: is the part of the processor that performs the reading and executing of instructions.

Multicore CPU: is a single component with two or more independent cores.

Bits & Bytes: The minimum storage unit in a computer is a byte. A byte is composed of eight bits.

0s and 1s are interpreted as digits in the binary number system and are called bits (binary digits).

An encoding scheme is a set of rules that govern how a computer translates characters, numbers, and symbols into data the computer can actually work with.

- A kilobyte (KB) is about 1,000 bytes.
- A megabyte (MB) is about 1 million bytes.
- A gigabyte (GB) is about 1 billion bytes.
- A terabyte (TB) is about 1 trillion bytes.



Memory: A computer's memory consists of an ordered sequence of bytes for storing programs as well as data that the program is working with. Every byte in the memory has a unique address.

Memory address	Memory content	
.	.	
.	.	
.	.	
2000	01000011	Encoding for character 'C'
2001	01110010	Encoding for character 'r'
2002	01100101	Encoding for character 'e'
2003	01110111	Encoding for character 'w'
2004	00000011	Encoding for number 3
.	.	

FIGURE 1.2 Memory stores data and program instructions in uniquely addressed memory locations.

The screen resolution specifies the number of pixels in horizontal and vertical dimensions of the display device. Pixels (short for "picture elements") are tiny dots that form an image on the screen.

The dot pitch is the amount of space between pixels, measured in millimeters. The smaller the dot pitch, the sharper the display.

Programming Languages

Machine language: A computer's native language is called machine language. It consists of binary instructions (0s and 1s) that the computer can directly understand and execute. For example, to add two numbers, you might have to write an instruction in binary form like this:

```
1101101010011010
```

Assembly Language: programs written in machine language are hard to read and modify. Assembly language uses a short descriptive word, known as a mnemonic, to represent each of the machine-language instructions

For example, add means addition and sub means subtraction. To add 2 and 3, we can write:
add 2, 3, result

Assembler is used to translate assembly-language programs into machine code. Assembly language is referred to as a low-level language, because assembly language is close in nature to machine language and is machine dependent.

High level language: In the 1950s, high-level programming languages were developed. These languages are platform independent, easy to learn, and use English-like statements. For example:

```
area = 5 * 5 * 3.14159;
```

This statement calculates the area of a circle with radius 5. Different high-level languages are designed for different purposes.

A program written in a high-level language is called *source code*. Since computers cannot directly execute source code, it must be translated into machine code using an interpreter or a compiler.

- **Interpreter:** Translates and executes one statement at a time.



- **Compiler:** Translates the entire program into machine code before execution.

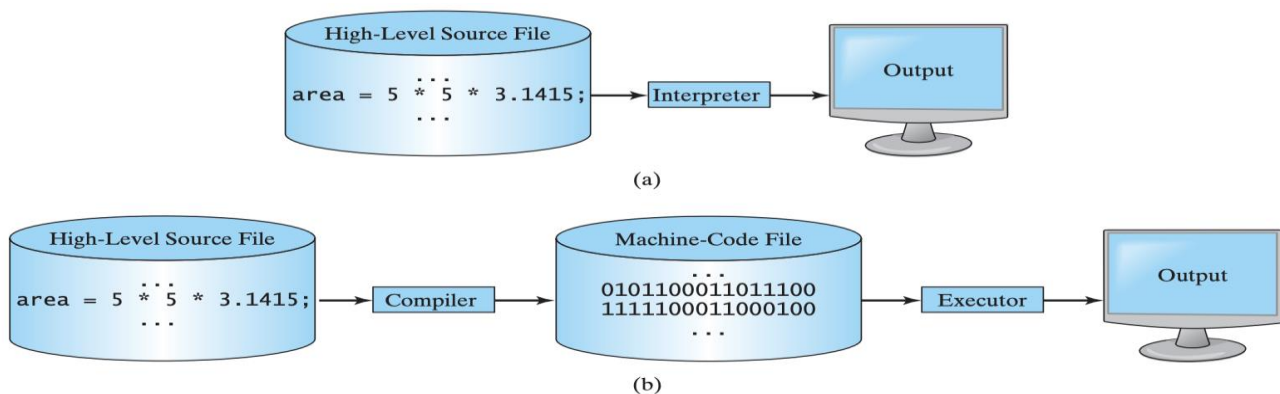


FIGURE 1.4 (a) An interpreter translates and executes a program one statement at a time. (b) A compiler translates the entire source program into a machine-language file for execution.

Operating System

- An Operating System (OS) is the most important software that manages and controls computer activities.
- Popular operating systems: Microsoft Windows, macOS, and Linux.
- Application software cannot run without an operating system.

Main Functions of an Operating System

1. **Controlling and Monitoring System Activities**
 - Handles input/output devices.
 - Manages files, folders, printers, and disks.
 - Provides security and prevents interference between programs/users.
2. **Allocating and Assigning Resources**
 - Allocates CPU time, memory, storage, and input/output devices to programs.
3. **Scheduling Operations**
 - Manages execution of programs efficiently.

Important Concepts

- **Multiprogramming:** Multiple programs share the CPU simultaneously.
- **Multithreading:** A single program performs multiple tasks at the same time.
- **Multiprocessing:** Two or more processors work together to complete tasks faster.

Java, the World Wide Web, and Beyond

- Java is a powerful and versatile programming language used for mobile, desktop, and server applications.
- Java was developed by James Gosling at Sun Microsystems in 1991.
- Originally called Oak, it was renamed Java in 1995.
- Oracle purchased Sun Microsystems in 2010.

Features of Java

- Platform independent: "Write Once, Run Anywhere."
- Simple and object-oriented.
- Secure, portable, and robust.



- Supports multithreading and high performance.

Uses of Java

- Web server applications
- Dynamic Web pages
- Desktop applications
- Mobile applications
- Server-side and distributed applications
- Android applications are mainly developed using Java.

World Wide Web & Internet

- The World Wide Web (WWW) is an information system accessed through the Internet.
- The Internet is the infrastructure that supports the Web.

Java Applets & Applications

- Java became popular because Java programs could run inside Web browsers.
- These browser-based Java programs are called applets.
- Applets use graphical components like buttons, text fields, and radio buttons for user interaction.
- Applets are embedded in HTML (HyperText Markup Language) files.
- HTML is used to create and link Web pages with images, sound, and video.

Rich Internet Applications (RIA)

- Java is used to develop Rich Internet Applications (RIA).
- RIAs provide features like desktop applications on the Web.

The Java Language Specification, API, JDK, and IDE

- Java syntax is defined in the Java Language Specification.
- The Java API (Application Program Interface) provides predefined classes and interfaces for Java programming.
- JDK (Java Development Kit) is a software used to develop and run Java programs.
- IDE (Integrated Development Environment) helps programmers write and develop programs quickly.

Key Points

- Java follows strict syntax rules.
- If syntax rules are not followed, the computer cannot understand the program.
- The Java API is also called the Java Library.
- The API keeps expanding with new classes and features.

Java Editions

Java SE: Used for client-side and standalone applications.

Java EE: Used for server-side and web applications like Servlets, JSP, and JSF.

Java ME: Used for mobile and small devices.

JDK

- JDK (Java Development Kit) is used to develop and test Java programs.
- Java SE 8 uses JDK 1.8 / JDK 8.



Popular Java IDEs: NetBeans, Eclipse, TextPad

Advantages of IDE

- Easy editing and debugging
- Compile and run programs quickly
- Provides graphical user interface (GUI)
- Online help support

A simple Java Program

- A Java program starts execution from the `main()` method.
- Console input means taking input from the keyboard.
- Console output means displaying output on the screen.

Listing 1.1 Welcome.java

```
1 public class Welcome {
2     public static void main(String[] args) {
3         System.out.println("Welcome to Java!");
4     }
5 }
```

Output

Welcome to Java!

Explanation:

- Line 1: `public class Welcome` → Defines a class named `Welcome`. Every Java program must have at least one class. By convention, class names begin with an uppercase letter.
- Line 2: `public static void main(String[] args)` → Defines the `main()` method. Program execution starts from the `main()` method, which is called the entry point of the program. A class may contain multiple methods.
- Method: A method is a construct that contains statements used to perform tasks.
- Line 3: `// Display message Welcome to Java! on the console` → This is a line comment. Comments are used to explain the program and are ignored by the compiler.

Types of Comments

Line Comment

```
// This application program displays Welcome to Java!
```

Block Comment

```
/* This application program displays Welcome to Java! */
/* This application program
displays Welcome to Java! */
```

- Line 4: `System.out.println("Welcome to Java!");` → Displays the string "Welcome to Java!" on the console.
- A string is a sequence of characters enclosed in double quotation marks " ".
- Every Java statement ends with a semicolon `;`, called the statement terminator.

Keywords / Reserved Words

- `public`, `class`, `static`, and `void` are reserved words (keywords).
- Keywords have special meanings to the compiler and cannot be used for other purposes.

Curly Braces { }

- Curly braces form a block in Java.
- { → Opening brace
- } → Closing brace

Types of Blocks

- **Class Block:** Groups data and methods of a class.
- **Method Block:** Groups statements inside a method.
- Blocks can be nested inside other blocks.

```
public class Welcome { ← Class block
    public static void main(String[] args) { ← Method block
        System.out.println("Welcome to Java!");
    }
}
```

Character	Name	Description
{ }	Opening and closing braces	Denote a block to enclose statements.
()	Opening and closing parentheses	Used with methods.
[]	Opening and closing brackets	Denote an array.
//	Double slashes	Precede a comment line.
" "	Opening and closing quotation marks	Enclose a string (i.e., sequence of characters).
;	Semicolon	Mark the end of a statement.

Creating, Compiling, and Executing a program

- A Java program is saved in a .java file and compiled into a .class file.
- The .class file is executed by the Java Virtual Machine (JVM).

The journey of any Java program begins with you, the programmer, writing the source code. Using a simple text editor (like Notepad) or an advanced IDE (like NetBeans or Eclipse), you write your program and save it to your disk as a file with a .java extension, such as Welcome.java. However, programming is rarely perfect on the first try, making this a continuous development cycle. As shown in Figure 1.6, you will often find yourself repeatedly writing, compiling, testing, and modifying this source code until the program runs flawlessly. Once your source code is ready, the computer cannot understand it directly because Java is a high-level language. This is where the Java Compiler steps in. Provided you have the Java Development Kit (JDK) installed, you can open your command window and type the command `javac Welcome.java`.

If the compiler catches any syntax mistakes, it will throw an error, forcing you back to the editing stage. But if your code is correct, the compiler successfully translates your source code into a new, low-level language format. It saves this output as a new file with a .class extension, such as Welcome.class.

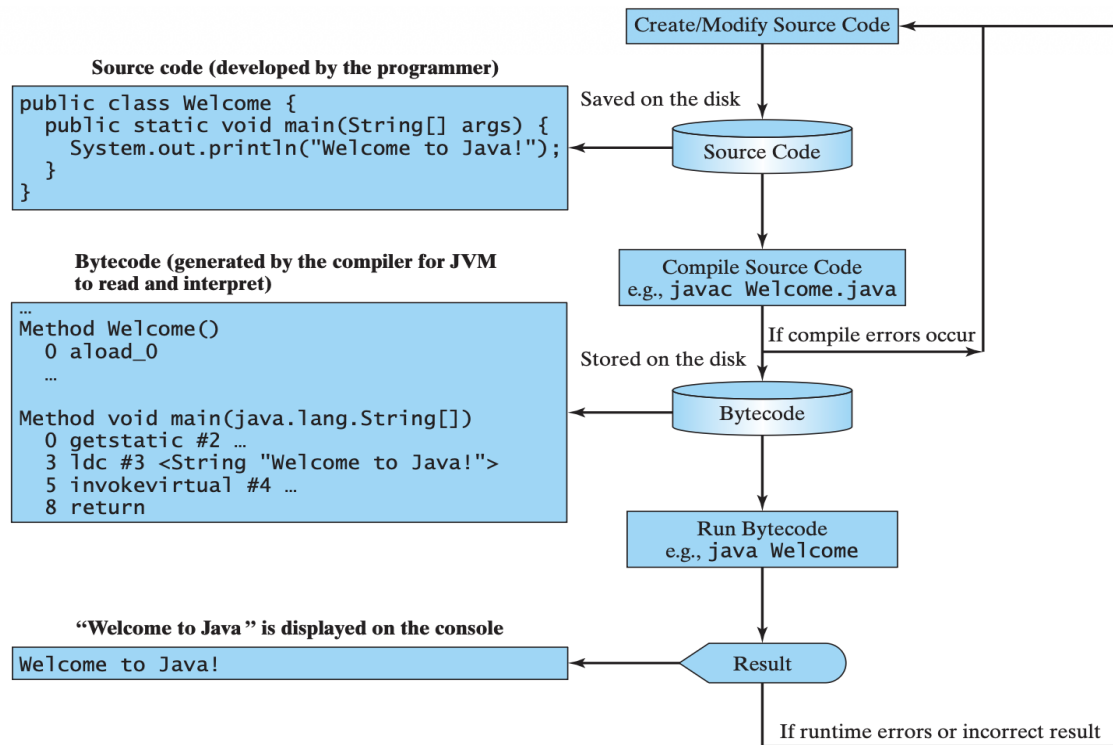


FIGURE 1.6 The Java program-development process consists of repeatedly creating/modifying source code, compiling, and executing programs.

This newly generated `.class` file does not contain standard machine code; instead, it contains *Java Bytecode*. This bytecode is the secret behind Java's biggest advantage. It is "architecture-neutral," meaning it isn't tied to any specific physical computer or hardware. As illustrated in Figure 1.8, instead of running directly on your computer's operating system, this bytecode is designed to run inside a special software environment called the *Java Virtual Machine (JVM)*. If a device has a JVM installed, it can run your Java bytecode perfectly.

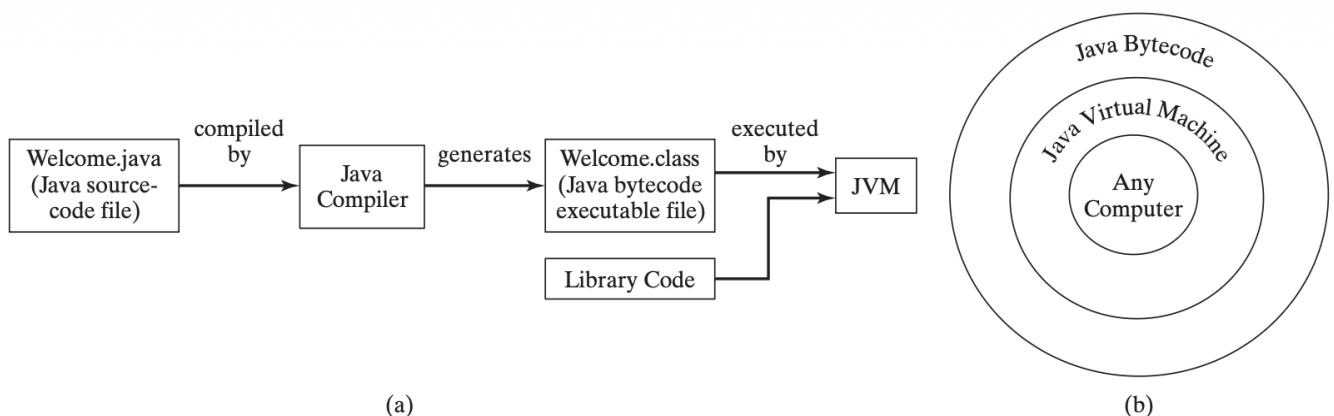


FIGURE 1.8 (a) Java source code is translated into bytecode. (b) Java bytecode can be executed on any computer with a Java Virtual Machine.

Now that you have your bytecode, you initiate the execution phase by typing the command `java Welcome` in your command window. But before the program actually starts doing its job, the JVM performs strict behind-the-scenes checks to keep your computer safe:

- First, the *Class Loader* steps in. It takes your `.class` bytecode from the disk and loads it into the computer's memory. If your code needs any external Java Library Code to function, the class loader dynamically pulls that in just in time.



- Next, the Bytecode Verifier takes over. It carefully inspects the loaded bytecode to ensure it hasn't been tampered with and that it strictly follows Java's security rules, guaranteeing it won't harm your system.

Once the bytecode is loaded and verified to be secure, the JVM finally begins execution. Acting as an Interpreter, the JVM reads the bytecode instructions one at a time. It translates each individual instruction into the target machine's native language and executes it immediately before moving on to the next one. This step-by-step translation and execution finally produce the result of your program, displaying it on your screen.

(If this result is incorrect or crashes, you hit a runtime error, and the cycle begins all over again: back to editing the .java file!)

Programming Style and Documentation

Good programming style and appropriate documentation are just as important as writing the code itself. Even if a program compiles and runs correctly, poor formatting (like writing everything on one line) makes it incredibly difficult to read. Good style and documentation reduce errors and make debugging and maintenance much easier.

1. Comments and Comment Styles

Comments are explanatory remarks that help programmers understand the code. They should be concise so they do not clutter the program.

- **Program Summaries:** Always include a summary at the beginning of a program explaining what it does, its key features, and any unique techniques used.
- **Step-by-Step Explanations:** In long programs, use comments to introduce major steps and explain complex logic.
- **Java Comment Types:**
 - Line Comments (`//`): Best used for explaining specific steps inside a method.
 - Block Comments (`/* ... */`): Used for longer, multi-line explanations.
 - Javadoc Comments (`/ ... */`): Used to document an entire class or method. These must be placed directly above the class or method header. A key feature of Javadoc comments is that they can be automatically extracted into an HTML documentation file using the JDK's `javadoc` command.

2. Proper Indentation and Spacing

Consistent spacing and indentation visually illustrate the structural relationships between various parts of your code.

- **Indentation:** Indent each nested subcomponent or statement by at least two spaces more than the construct it sits inside. While Java can read a program written on a single line, humans need alignment to read and maintain it.
- **Spacing Around Operators:** Always add a single space on both sides of a binary operator for clarity.
 - Bad Style: `System.out.println(3+4*4);`
 - Good Style: `System.out.println(3 + 4 * 4);`

3. Block Styles

A block is a group of statements surrounded by curly braces `{}`. There are two primary styles for formatting these blocks. You should pick one and use it consistently throughout your code rather than mixing them.



- **Next-line Style:** The opening brace is placed on the line immediately following the header, aligning the opening, and closing braces vertically. This prioritizes readability.

```
1 public class Test
2 {
3     public static void main(String[] args)
4     {
5         System.out.println("Block Styles");
6     }
7 }
```

- **End-of-line Style:** The opening brace is placed at the end of the header line. This saves vertical space and is the standard style used in the official Java API source code.

```
1 public class Test{
2     public static void main(String[] args){
3         System.out.println("Block Styles");
4     }
5 }
```

Programming Errors

In Java, when your program doesn't work as expected, the issue usually falls into one of three categories: Syntax Errors, Runtime Errors, or Logic Errors. Understanding the difference between them is the key to fixing your code quickly.

1. Syntax Errors (The "Grammar" Mistakes)

Think of syntax errors like making a grammatical mistake in a sentence. Also known as compile errors, these happen when you break Java's coding rules. This could be mistyping a keyword, forgetting punctuation, or opening a brace { without closing it }.

Here is an example of a program (ShowSyntaxErrors.java) with syntax errors:

```
1 public class ShowSyntaxErrors {
2     public static main(String[] args) {
3         System.out.println("Welcome to Java");
4     }
5 }
```

- **What's wrong here?** 1. In line 2, the keyword void is missing before main. 2. In line 3, the string "Welcome to Java" is missing its closing quotation mark.
- **How to fix it:** The compiler will refuse to run this and will point out the errors. A great habit is to fix errors from the top line downward, as fixing an early error (like a missing brace or semicolon) often clears up a cascade of fake errors below it.

2. Runtime Errors (The "Crash")

Sometimes your code's syntax is perfectly fine, and the compiler gives it a green light. But when you run the program, it crashes and terminates abnormally. This is a runtime error.

- **Why it happens:** The environment (JVM) encounters an impossible operation while the program is actively running.

Here is an example (ShowRuntimeErrors.java):

```
1 public class ShowRuntimeErrors {
2     public static void main(String[] args) {
3         System.out.println(1 / 0);
4     }
5 }
```



5 }

- **What's wrong here?** The syntax is mathematically correct to the compiler, but you cannot divide a number by zero. When the program reaches line 3 during execution, it will crash and terminate abnormally.
- **Common Causes:** Aside from mathematical impossibilities like dividing by zero, these often happen due to bad user input (e.g., the user types the word "Ten" when the program asked them to type the number 10).

3. Logic Errors (The "Silent Bugs")

These are the trickiest errors to catch. Your code has perfect syntax, and it doesn't crash while running. However, it does not do what you intended it to do.

- **The Problem:** The compiler and JVM don't know what your goal was, so they won't flag any errors.

Here is an example (`ShowLogicErrors.java`) that tries to convert 35° Celsius to Fahrenheit:

```
1 public class ShowLogicErrors {
2     public static void main(String[] args) {
3         System.out.println("Celsius 35 is Fahrenheit degree ");
4         System.out.println((9 / 5) * 35 + 32);
5     }
6 }
```

- **What's wrong here?** If you run this, it will print 67 instead of the correct answer, 95.0. Why? In Java, when you divide two integers like `9 / 5`, it truncates the decimal and just gives you 1.
- **How to fix it:** To get the correct math, you need to use decimal numbers to force floating-point division: `9.0 / 5`. Finding these bugs requires careful testing and tracing of your code.

4. Common Errors

Common Error 1: Missing Braces `{}` In Java, braces group your code into blocks. Every opening brace must have a closing brace.

Pro-Tip: Type the closing brace the exact second you type the opening one.

```
public class Welcome {
} // Type this closing brace right away to match the opening brace!
```

Common Error 2: Missing Semicolons `;` Every standard statement in Java must end with a semicolon (like a period at the end of an English sentence). It is easy to forget this on the last line of a block.

```
1 public static void main(String[] args) {
2     System.out.println("Programming is fun!");
3     System.out.println("Fundamentals First");
4     System.out.println("Problem Driven") // ERROR: Missing a semicolon here!
5 }
```

Common Error 3: Missing Quotation Marks `"` Whenever you are printing text, it must be completely wrapped in quotes. Forgetting the closing quote will break the code.

```
1 public class Test {
2     // ERROR: 'Main' should be 'main', and 'string' should be 'String'
3     public static void Main(string[] args) {
4         System.out.println("Problem Driven "); // ERROR: Missing the
closing quotation mark!
5     }
```



```
6 }
```

```
System.out.println("Problem Driven "); // ERROR: Missing the closing quotation mark!
```

Common Error 4: Misspelling Names (Case-Sensitivity) Java is strictly case-sensitive. Main and main are two entirely different things to the computer.

```
1 public class Test {
2     // ERROR: 'Main' should be 'main', and 'string' should be 'String'
3     public static void Main(string[] args) {
4         System.out.println((10.5 + 2 * 3) / (45 - 3.5));
5     }
6 }
```